

Home

Library

Profile

Stories

Stats

Following

Dev Genius

Python in Plain En...

The Useful Tech

Generative AI

Level Up Coding

AI Mind

Top Python Librari...

Deep concept

AI Quick Tips

Realworld AI Use C...

More

Member-only story

Best SQL Hacks I Wouldn't Have Believed If I Hadn't Used Them Myself



Anchit Gupta

Follow

7 min read · Feb 7, 2026



535



9



A Data Engineer's collection of jaw-dropping SQL tricks that changed how I write queries forever.

[\[Non-Members Link\]](#)



I've been writing SQL for over a decade. I thought I knew the language inside and out — until I stumbled upon these techniques. Each one made me pause, re-read, test it three times, and then wonder why nobody taught me this sooner.

This isn't your typical “use indexes” advice. These are real, battle-tested hacks that have saved me hours of work, thousands of lines of code, and more than a few headaches in production.

Let's dive in.

1. VALUES as an Inline Table (No Actual Table Needed)

Most people think you need a physical table or a CTE to work with a small dataset. You don't.

```
SELECT *  
FROM (VALUES  
      (1, 'Alice', 'Engineering'),  
      (2, 'Bob', 'Marketing'),  
      (3, 'Carol', 'Engineering')  
    ) AS employees(id, name, department)  
WHERE department = 'Engineering';
```

Why this blew my mind: You can create a throwaway table *right inside your query*. It's perfect for quick lookups, mapping values, or mocking data during development without ever touching the database schema.

Real-world use: I once needed to map 15 status codes to human-readable labels. Instead of creating a mapping table and getting a DBA approval, I just inlined the mapping with `VALUES` and joined it to my main query.

2. FILTER Clause — The Elegant Alternative to CASE WHEN Inside Aggregates

If you've ever written something like this, you know the pain:

```
-- The old, ugly way
SELECT
  department,
  COUNT(CASE WHEN status = 'active' THEN 1 END) AS active_count,
  COUNT(CASE WHEN status = 'inactive' THEN 1 END) AS inactive_count,
  SUM(CASE WHEN status = 'active' THEN salary END) AS active_salary
FROM employees
GROUP BY department;
```

Now look at this:

```
-- The clean way (PostgreSQL, SQLite 3.30+, DuckDB)
SELECT
  department,
  COUNT(*) FILTER (WHERE status = 'active') AS active_count,
  COUNT(*) FILTER (WHERE status = 'inactive') AS inactive_count,
  SUM(salary) FILTER (WHERE status = 'active') AS active_salary
FROM employees
GROUP BY department;
```

Why this blew my mind: It reads like English. “Count everything, *filtered where* status is active.” It’s cleaner, less error-prone, and drastically improves readability when you have 10+ conditional aggregations in a dashboard query.

3. Generate an Entire Date Series from Thin Air

Ever needed a row for every single date in a range, even dates with no data? Most people write a loop in Python. Don't.

```
-- PostgreSQL
SELECT generate_series(
  '2024-01-01'::date,
  '2024-12-31'::date,
```

```

        '1 day'::interval
    )::date AS date;

-- MySQL 8.0+ (recursive CTE approach)
WITH RECURSIVE dates AS (
    SELECT '2024-01-01' AS date
    UNION ALL
    SELECT date + INTERVAL 1 DAY
    FROM dates
    WHERE date < '2024-12-31'
)
SELECT * FROM dates;

```

Why this blew my mind: Left-join this date series to your actual data, and you'll never have “missing date” gaps in your time-series reports again. Your dashboards will thank you.

```

SELECT
    d.date,
    COALESCE(o.order_count, 0) AS order_count
FROM generate_series('2024-01-01'::date, '2024-01-31'::date, '1 day') AS d(
LEFT JOIN (
    SELECT order_date, COUNT(*) AS order_count
    FROM orders
    GROUP BY order_date
) o ON o.order_date = d.date
ORDER BY d.date;

```

No more “why is January 17th missing from the chart?” conversations.

4. GROUPING SETS — Multiple GROUP BYs in a Single Pass

Imagine your manager asks: “I need totals by department, totals by region, and a grand total. Can you write three queries?”

No. You write one.

```

SELECT
    department,

```

```

        region,
        SUM(revenue) AS total_revenue
FROM sales
GROUP BY GROUPING SETS (
    (department),
    (region),
    (department, region),
    () -- grand total
);

```

Sample output:

```

department region total_revenue Engineering NULL 500000 Marketing
NULL 300000 NULL East 450000 NULL West 350000 Engineering East
280000 Engineering West 220000 NULL NULL 800000

```

Why this blew my mind: One table scan. One query. Multiple levels of aggregation. The database optimizer handles it far more efficiently than running three separate queries. Pair it with `GROUPING()` function to distinguish real NULLs from "this is a subtotal" NULLs.

5. Lateral Joins — The “For Each” Loop of SQL

This one is severely underused. A `LATERAL` join lets the subquery on the right *reference* columns from the left side — essentially iterating per row.

Problem: For each department, get the top 3 highest-paid employees.

```

-- PostgreSQL / MySQL 8.0.14+
SELECT
    d.department_name,
    top_emp.name,
    top_emp.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT name, salary
    FROM employees e
    WHERE e.department_id = d.id
    ORDER BY salary DESC
    LIMIT 3
) AS top_emp;

```

Why this blew my mind: Before `LATERAL`, solving "top-N per group" required window functions with CTEs or correlated subqueries. Lateral joins express this so naturally it almost feels like cheating. SQL Server users know this as `CROSS APPLY` / `OUTER APPLY`.

6. The `::` Operator and String-to-Array Magic

In PostgreSQL, you can split strings, unnest arrays, and reassemble them — all in pure SQL.

```
-- Split a comma-separated string into rows
SELECT unnest(string_to_array('apple,banana,cherry', ',')) AS fruit;
```

fruit apple banana cherry

Now the reverse — collapse rows back into a single value:

```
SELECT
    department,
    STRING_AGG(employee_name, ',' ORDER BY employee_name) AS team_members
FROM employees
GROUP BY department;
```

department team_members Engineering Alice, Bob, Carol Marketing
Dave, Eve.

Why this blew my mind: I used to export data to Python just to split and rejoin strings. Turns out SQL has been able to do this natively for years. Combined, these functions let you normalize denormalized CSV columns and denormalize normalized rows — without ever leaving your

query.

7. Window Functions with RANGE vs ROWS — They Are NOT the Same

Most tutorials teach `ROWS BETWEEN`. Few explain when `RANGE BETWEEN` saves your life.

```
-- Calculate a 7-day rolling average of daily revenue
SELECT
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW
    ) AS rolling_7day_avg
FROM daily_sales;
```

The critical difference:

- `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` looks at the previous 6 *rows*, regardless of date gaps.
- `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW` looks at the previous 6 *calendar days*, correctly handling missing dates.

If your data has gaps (weekends, holidays), `ROWS` gives you wrong averages. `RANGE` gives you correct ones.

Why this blew my mind: I debugged a “slightly off” rolling average for two days before realizing the issue was `ROWS` vs `RANGE`. A one-word change fixed everything.

8. INSERT ... ON CONFLICT (Upsert) — Goodbye Delete-Then-Insert

The old pattern of “delete existing, then insert new” is fragile and not

atomic. Enter the upsert:

```
-- PostgreSQL
INSERT INTO user_settings (user_id, theme, language)
VALUES (42, 'dark', 'en')
ON CONFLICT (user_id)
DO UPDATE SET
    theme = EXCLUDED.theme,
    language = EXCLUDED.language;

-- MySQL equivalent
INSERT INTO user_settings (user_id, theme, language)
VALUES (42, 'dark', 'en')
ON DUPLICATE KEY UPDATE
    theme = VALUES(theme),
    language = VALUES(language);
```

Why this blew my mind: It's atomic, it's one statement, and it handles the “insert if new, update if exists” pattern that shows up in *every single ETL pipeline*. No more race conditions. No more try-catch blocks in your application code.

9. DISTINCT ON — PostgreSQL's Secret Weapon

This is, hands down, my favorite PostgreSQL-specific feature.

Problem: Get the most recent order for each customer.

```
-- The typical (verbose) approach
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)
    FROM orders
)
SELECT * FROM ranked WHERE rn = 1;

-- The PostgreSQL shortcut
SELECT DISTINCT ON (customer_id)
    customer_id, order_id, order_date, total
FROM orders
ORDER BY customer_id, order_date DESC;
```

Same result, half the code, and often faster because the optimizer can use an index scan directly.

Why this blew my mind: Every “get the latest/first per group” problem becomes a two-line solution. I use it almost daily.

10. Using CTEs as “Writeable” Stages (Data-Modifying CTEs)

Most people know CTEs for reading data. Fewer know you can use them to *write* data and chain operations together.

```
-- Archive old orders and delete them in one atomic statement
WITH archived AS (
    INSERT INTO orders_archive
    SELECT * FROM orders WHERE order_date < '2023-01-01'
    RETURNING *
),
deleted AS (
    DELETE FROM orders
    WHERE order_id IN (SELECT order_id FROM archived)
    RETURNING *
)
SELECT
    COUNT(*) AS rows_processed
FROM deleted;
```

Why this blew my mind: One statement. Atomic. No temporary tables. No multi-step scripts. The `RETURNING` clause feeds the output of one operation into the next. This is how you write ETL logic that doesn't break at 3 AM.

Bonus: The `EXPLAIN ANALYZE` Mindset

This isn't a “hack” — it's a habit. But it will make you a 10x better SQL developer.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT ...
```

Read the output bottom-to-top. Look for:

- **Seq Scan** on large tables (you probably need an index)
- **Nested Loop** with high row counts (consider a hash join via restructuring)
- **Actual rows vs Estimated rows** being wildly different (stale statistics — run `ANALYZE`)

The mindset shift: Stop guessing why your query is slow. *Ask the database to tell you.* Every senior data engineer I know has `EXPLAIN ANALYZE` practically tattooed on their forearm.

Wrapping Up

SQL is deceptively deep. I've been writing it for years, and these tricks still surprise me with how much cleaner and faster they make my work. Here's the cheat sheet:

The best part? Most of these work across PostgreSQL, MySQL 8+, SQL Server, and DuckDB with minor syntax adjustments. You don't need a new tool — you just need to dig deeper into the one you already have.

If even one of these saves you an hour this week, this article did its job.

If you found this useful, give it a clap and follow for more data engineering deep dives. Got a SQL trick that blew your mind? Drop it in the comments — I'm always learning too.

[Data Science](#)[Data Engineering](#)[Data](#)[Programming](#)[Clean Code](#)



Written by Anchit Gupta

409 followers · 23 following

Follow

Responses (9)



Pancake Agree

What are your thoughts?

See all responses